

# 智能算法及应用

## 10. 价值学习方法

智能算法及应用@华南理工大学2025

# 价值学习

$$Q^*(s_t, a_t) = \max_{\pi} Q_{\pi}(s_t, a_t), \forall s_t \in \mathcal{S}, a_t \in \mathcal{A}$$

智能算法及应用@华南理工大学2025



$$\pi(s) = \operatorname{argmax}_{a \in \mathcal{A}} Q^*(s, a)$$

(为了方便描述，下面我们将 $Q^*$ 函数简记为 $Q$ )

# Q-Learning

1.

Q-learning是一种离线策略 (off-policy) 的时序差分 (TD) 方法:

2.

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[ R(s, a) + \gamma \max_{A_{t+1}} Q(S_{t+1}, A_{t+1}) - Q(s, a) \right]$$

3.

传统的Q-Learning用表格形式表示Q值, 如下图所示。

|             | 第 1 种<br>动作 | 第 2 种<br>动作 | 第 3 种<br>动作 | 第 4 种<br>动作 |
|-------------|-------------|-------------|-------------|-------------|
| 第 1 种<br>状态 | 380         | -95         | 20          | 173         |
| 第 2 种<br>状态 | -7          | 64          | -195        | 210         |
| 第 3 种<br>状态 | 152         | 72          | 413         | -80         |

# Q-Learning

对于Q-Learning中的每个episode，每走一步迭代更新一次 $Q$ 值表：

- 首先我们基于状态  $s$ ，用  $\epsilon$ -贪婪法选择到动作  $a$ ；
- 然后执行动作  $a$ ，得到奖励  $r$ ，并进入状态  $s'$ ；
- 最后基于状态  $s'$ ，使用贪婪法选择  $a'$ ，也就是说，选择状态  $s'$  下使得  $Q(s', a')$  最大的  $a'$  来更新价值函数。

|             | 第 1 种<br>动作 | 第 2 种<br>动作 | 第 3 种<br>动作 | 第 4 种<br>动作 |
|-------------|-------------|-------------|-------------|-------------|
| 第 1 种<br>状态 | 380         | -95         | 20          | 173         |
| 第 2 种<br>状态 | -7          | 64          | -195        | 210         |
| 第 3 种<br>状态 | 152         | 72          | 413         | -80         |

注：此时，如果是采用同策略的SARSA算法，会继续基于状态  $s'$ ，用  $\epsilon$ -贪婪法选择  $a'$ ，然后来更新价值函数。但是Q-Learning不同，当  $s'$  确定后，可以认为  $a'$  也能跟着确定了。

另外Q-Learning中的动作  $a'$  只会参与价值函数的更新，不会真正的执行（价值函数更新后，新的执行动作需要基于状态  $s'$ ，用  $\epsilon$ -贪婪法重新选择得到）。这一点也和SARSA稍有不同。对于SARSA，价值函数更新使用的  $a'$  会作为下一阶段开始时候的执行动作。

# Q-Learning

Initialize  $Q(s, a)$  arbitrarily

Repeat (for each episode):

Initialize  $s$

Repeat (for each step of episode):

Choose  $a$  from  $s$  using policy derived from  $Q$  (e.g.,  $\epsilon$ -greedy)

Take action  $a$ , observe  $r, s'$

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

$s \leftarrow s'$ ;

until  $s$  is terminal

# 深度Q网络 (DQN)

前面介绍的 Q-learning 需要记录  $(s, a)$  对应的  $Q$  值, 当我们所处理的问题具有较大的状态集和动作集时, 表格型的方法就显得十分的低效了。此时我们需要一种新的模型方法来处理这种问题。所以出现了结合了神经网络的 Q-Learning 方法——Deep Q Network (DQN), 通过在探索的过程中训练网络, 最后所达到的网络将当前状态输入, 输出它的每一个可能动作的值函数, 也即  $f(s; w) = Q(s, a)$ . 这个  $f$  就是训练的网络, 它的参数由  $w$  表示。



这里注意到：动作集合有多少个动作，网络就需要多少个输出节点，隐含了我们的动作是有限个的离散动作。

# 深度Q网络 (DQN)

从下面开始我们将DQN中的网络记为 $Q(s, a; \omega)$ 。

要训练这个网络，我们需要定义损失函数：

$$L(\omega) = \frac{1}{2} [Q(s, a; \omega) - y]^2$$

由于使用了时序差分方法， $y$  是 TD 目标。

$$y = r + \gamma \max_{a'} Q(s', a'; \omega)$$

尽管它也依赖于  $\omega$ ，但我们假设其是个常数。随后计算  $L$  关于  $\omega$  的梯度：

$$\nabla_{\omega} L(\omega) = \underbrace{(Q(s, a; \omega) - y)}_{\text{TD误差}} \cdot \nabla_{\omega} Q(s, a; \omega)$$

随后用梯度下降来更新网络参数。

# 深度Q网络 (DQN)

基本步骤为:

1. 对于一个transition四元组, 记作  $(s, a, s', r)$ 。
2. 用 DQN 做前向传播, 得到  $Q$  值:

$$q = Q(s, a; \omega) \text{ 和 } q' = \max_a Q(s', a'; \omega)$$

3. 计算 TD 目标和 TD 误差:

$$y = r + \gamma \cdot q' \text{ 和 } \delta = q - y$$

4. 做梯度下降更新 DQN 参数:

$$\omega \leftarrow \omega - \alpha \cdot \delta \cdot \nabla_{\omega} Q(s, a; \omega)$$

其中 $\alpha$ 是学习率。



# 深度Q网络 (DQN)

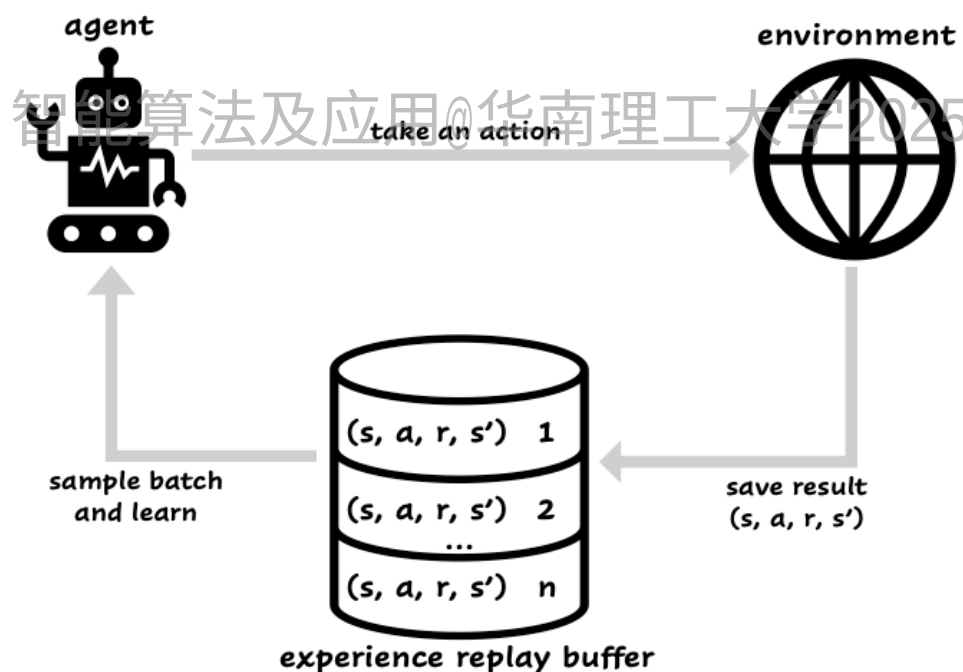
DQN结合深度学习和Q-Learning的方法，在概念十分直观，但仍面临着挑战：

- 深度学习需要大量带标签的训练数据；强化学习从环境的反馈中进行学习，但是奖励经常是稀疏、带噪声、延时的；
- 深度学习假设样本数据是独立同分布 (independent and identically distributed, i.i.d.) 的，但是强化学习是一个序列决策过程，状态之间存在很强的关联性。

下面我们介绍DQN的两项核心技术，用于克服上述困难。

# 经验回放 (Experience Replay)

DQN 采用**经验回放 (Experience Replay)** 机制，将训练过的数据进行储存到 Replay Buffer 中，以便后续从中随机采样进行训练，好处就是：1. 数据利用率高；2. 减少连续样本的相关性，从而减小方差。



# 经验回放 (Experience Replay)

DQN 采用**经验回放 (Experience Replay)** 机制, 将训练过的数据进行储存到 Replay Buffer 中, 以便后续从中随机采样进行训练, 好处就是: 1. 数据利用率高; 2. 减少连续样本的相关性, 从而减小方差。

具体地, 经验回放的一个基本单元是transition, 一个包含  $(s, a, s', r)$  的元组, 即 (状态, 动作, 状态执行动作后到达的下一个状态, 这一步状态转移得到的奖励), 也就是关于智能体走一步所发生的信息。通过记录一段连续时间的transition (经验), 积累够一定数量的 transition 后网络才开始更新, 在网络更新时从中随机抽取一批来计算 loss 训练网络, 打破数据间的关联。

DQN初始化容量为  $N$  的空Replay Buffer, 对每一个回合, 随机一个初始状态; 在接下来的  $T$  步中, 对每一步  $t$ : 将状态  $s_t$  给到网络 得到估计的每个动作对应的 $Q$ 值, 通过 $\epsilon$ -greedy 选择一个动作  $a_t$  并获得下一个状态  $s_{t+1}$  和奖励  $r_t$ , 将这一个 transition存入 Replay Buffer。

# 经验回放 (Experience Replay)

NIPS 2013论文中 DQN 的伪代码:

---

**Algorithm 1** Deep Q-learning with Experience Replay

---

Initialize replay memory  $\mathcal{D}$  to capacity  $N$

Initialize action-value function  $Q$  with random weights

**for** episode = 1,  $M$  **do**

    Initialize sequence  $s_1 = \{x_1\}$  and preprocessed sequence  $\phi_1 = \phi(s_1)$

**for**  $t = 1, T$  **do**

        With probability  $\epsilon$  select a random action  $a_t$

        otherwise select  $a_t = \max_a Q^*(\phi(s_t), a; \theta)$

        Execute action  $a_t$  in emulator and observe reward  $r_t$  and image  $x_{t+1}$

        Set  $s_{t+1} = s_t, a_t, x_{t+1}$  and preprocess  $\phi_{t+1} = \phi(s_{t+1})$

        Store transition  $(\phi_t, a_t, r_t, \phi_{t+1})$  in  $\mathcal{D}$

        Sample random minibatch of transitions  $(\phi_j, a_j, r_j, \phi_{j+1})$  from  $\mathcal{D}$

        Set  $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$

        Perform a gradient descent step on  $(y_j - Q(\phi_j, a_j; \theta))^2$  according to equation 3

**end for**

**end for**

---

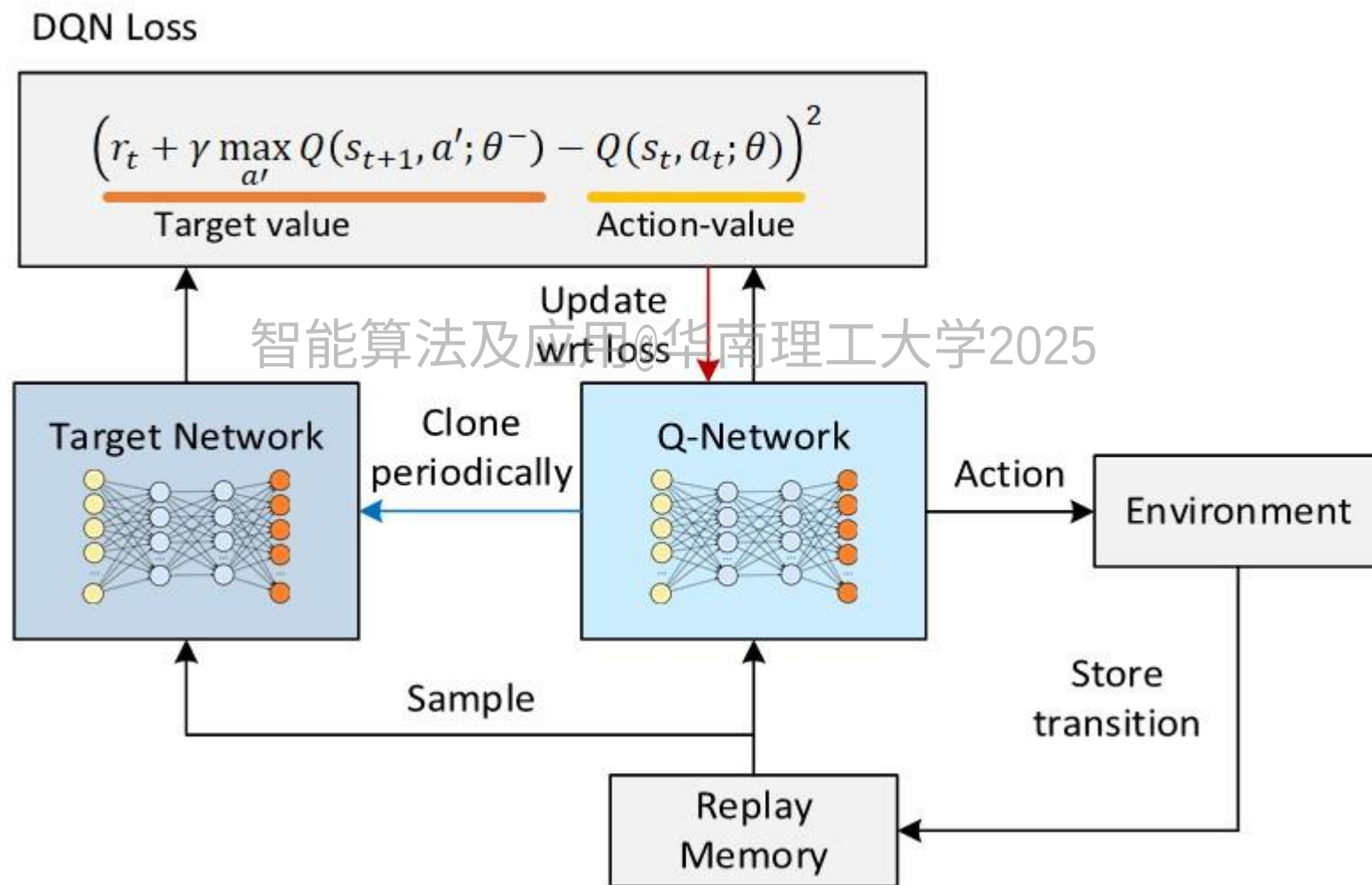
# 目标网络 (Target Network)

DQN中的TD 目标  $y = r + \gamma \max_{a'} Q(s', a'; \omega)$ , 然后做梯度下降更新  $\omega$ , 使得  $Q(s, a; \omega)$  更接近  $y$ 。但是TD目标本身用了同一个 $Q$ 值预测网络, 这会导致过估计问题, 导致网络训练震荡甚至发散。例如, 在动作  $(s_t, a_t)$  上的更新使得  $Q$  值增加, 此时  $s_t$  和  $s_{t+1}$  的相似性可能会导致所有动作  $a$  的  $Q(s_{t+1}, a)$  值增加, 从而使得由  $Q$  网络产生的训练目标值被过估计。

智能算法及应用@华南理工大学2025  
解决的途径是切断自举, 用另一个神经网络计算 TD 目标, 而不是用 DQN 自己。另一个神经网络就被称作**目标网络 (Target Network)**。把目标网络记作  $Q(s, a; \omega^-)$ , 它的神经网络结构与 DQN 完全相同, 但是参数  $\omega^-$  不同于  $\omega$ 。

具体而言, 目标网络每  $C$  步将通过直接复制 (硬更新) 或者指数衰减平均 (软更新) 的方式与主  $Q$  网络同步。通过使用旧参数生成 TD 目标, 使目标值的产生不受最新参数的影响, 从而大大减少发散和震荡的情况。

# 目标网络 (Target Network)



# 目标网络 (Target Network)

Nature 2015论文中 DQN伪代码:

**Algorithm 1: deep Q-learning with experience replay.**

Initialize replay memory  $D$  to capacity  $N$

Initialize action-value function  $Q$  with random weights  $\theta$

Initialize target action-value function  $\hat{Q}$  with weights  $\theta^- = \theta$

**For** episode = 1,  $M$  **do**

Initialize sequence  $s_1 = \{x_1\}$  and preprocessed sequence  $\phi_1 = \phi(s_1)$

**For**  $t = 1:T$  **do**

With probability  $\epsilon$  select a random action  $a_t$

otherwise select  $a_t = \operatorname{argmax}_a Q(\phi(s_t), a; \theta)$

Execute action  $a_t$  in emulator and observe reward  $r_t$  and image  $x_{t+1}$

Set  $s_{t+1} = s_t, a_t, x_{t+1}$  and preprocess  $\phi_{t+1} = \phi(s_{t+1})$

Store transition  $(\phi_t, a_t, r_t, \phi_{t+1})$  in  $D$

Sample random minibatch of transitions  $(\phi_j, a_j, r_j, \phi_{j+1})$  from  $D$

Set  $y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$

Perform a gradient descent step on  $(y_j - Q(\phi_j, a_j; \theta))^2$  with respect to the network parameters  $\theta$

Every  $C$  steps reset  $\hat{Q} = Q$

**End For**

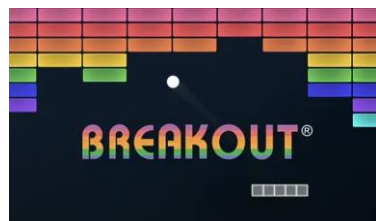
**End For**



# 深度Q网络（DQN）

DeepMind在论文中对这些技术的作用作了实验分析，在Atari游戏的测试中，使用经验回放和目标网络显著地提升了DQN总体的性能。

| 游戏名称           | 使用回放缓存和目标 Q 网络 | 使用回放缓存，且不使用目标 Q 网络 | 不使用回放缓存，使用目标 Q 网络 | 不使用回放缓存和目标 Q 网络 |
|----------------|----------------|--------------------|-------------------|-----------------|
| Breakout       | 316.8          | 240.7              | 10.2              | 3.2             |
| Enduro         | 1006.3         | 831.4              | 141.9             | 29.1            |
| River Raid     | 7446.6         | 4102.8             | 2867.7            | 1453.0          |
| Seaquest       | 2894.4         | 822.6              | 1003.0            | 275.8           |
| Space Invaders | 1088.9         | 826.3              | 373.2             | 302.0           |





# DQN的进一步改进

| 算法名称               | 核心创新                         | 解决的关键问题                             |
|--------------------|------------------------------|-------------------------------------|
| 优先经验回放             | 根据TD误差的绝对值对经验样本设定采样优先级       | 均匀采样导致低效学习                          |
| Double DQN         | 解耦动作选择与价值评估：用主网络选动作，目标网络评估价值 | Q值过估计（因最大化操作导致误差累积）                 |
| Dueling DQN        | 网络结构拆分为状态价值V和优势函数A           | 难以区分状态价值和动作优势                       |
| Multi-Step DQN     | 多步回报（使用n步TD误差）               | 单步TD更新引入短视偏差                        |
| Noisy DQN          | 在网络权重中注入参数化噪声                | $\epsilon$ -greedy探索效率低下（随机动作与环境无关） |
| Distributional DQN | 预测Q值的分布而非期望值                 | 传统Q-learning无法表达回报的随机性              |
| Rainbow            | 集成以上6种方法                     | 单一改进的局限性与潜在优势互补                     |

# 优先经验回放 (Prioritized Experience Replay, PER)

注意到在经验回放池里面的不同的样本由于TD误差的不同, 对我们反向传播的作用是不一样的。TD误差越大, 那么对我们反向传播的作用越大。

如果TD误差较大的样本更容易被采样, 则我们的算法会比较容易收敛。于是PER使用如下方法计算某个transition  $i$  的采样概率:

智能算法及应用@华南理工大学2025

$$P_i = \frac{p_i^\alpha}{\sum_k p_k^\alpha}$$

其中 $\alpha$  是一个指数超参数,  $\alpha = 0$  对应均匀采样情况;  $p_i$  指transition  $i$  的优先级, 比如可以通过如下方案计算:

- 方案一:  $p_i = |\delta_i| + \epsilon$ , 其中 $\delta_i$  是transition  $i$  的 TD 误差, 而  $\epsilon$  是一个是个很小的数, 保证所有样本都以非零的概率被抽到;
- 方案二:  $p_i = \frac{1}{\text{rank}(i)}$ , 其中  $\text{rank}(i)$  是transition  $i$  基于  $|\delta_i|$  的排序。

# 优先经验回放 (Prioritized Experience Replay, PER)

注意到PER中的抽样是非均匀的, 不同的样本有不同的抽样概率, 这样会导致 DQN 的预测有偏差。之前我们的损失函数是

$$L(\omega) = \frac{1}{N} \sum_{j=1}^N (Q(s_j, a_j; \omega) - y_j)^2$$

其中 $N$ 为经验回放池大小。为了矫正这个偏差, 我们应该给予不同的样本不同的权重系数, 即定义新的损失函数为:

$$L(\omega) = \sum_{j=1}^N w_j (Q(s_j, a_j; \omega) - y_j)^2$$

其中 $w_j$ 是一个重要性采样系数

$$w_j = \left( \frac{1}{N} \cdot \frac{1}{P_j} \right)^\beta$$

$\beta \in (0, 1)$ 是个需要调的超参数, 论文里建议一开始让  $\beta$  比较小, 最终增长到1。

# 优先经验回放 (Prioritized Experience Replay, PER)

## 超参数 $\beta$ 的作用

直接使用  $w_j = \frac{1}{NP_j}$  会导致权重过大 (尤其对  $P_j$  很小的低频样本), 造成训练不稳定 (高方差)。因此引入  $\beta$ :

- $\beta$  的作用: 通过幂次衰减 ( $w_j = (1/(NP_j))^\beta$ ) 控制权重幅度, 平滑高低权重差异, 平衡 **偏差 (校正程度)** 和 **方差 (训练稳定性)**。
  - 当  $\beta \rightarrow 0$ : 权重  $w_j \rightarrow 1$ , 即放弃校正 (等同于均匀采样), **高偏差, 低方差**。
  - 当  $\beta \rightarrow 1$ : 权重  $w_j = 1/(NP_j)$ , **完全校正偏差**, 但可能因某些  $P_j$  过小导致高方差。

## $\beta$ 从低逐渐调至1的动机

- **训练初期**: Q 网络估计不准确, 高优先级样本 (大 TD 误差) 可能仅是噪声信号, 过早完全校正 ( $\beta = 1$ ) 会引入 **不可靠梯度的振荡**。
  - 此时 **低  $\beta$**  (如 0.4~0.6) 弱化校正, 优先降低方差, 保持稳定性。
- **训练后期**: Q 网络更准确, 优先级真实反映样本重要性, 需强化校正以消除偏差。
  - **逐渐增大  $\beta$  至1**, 逐步逼近无偏估计, 同时模型已足够稳定适应方差。

# 优先经验回放 (Prioritized Experience Replay, PER)

在损失函数中引入重要性权重对应的是在随机梯度下降中对学习率的调节:

$$\alpha_j = w_j \cdot \alpha$$

注: 如果样本  $(s_j, a_j, r_j, s_{j+1})$  很重要, 它被抽到的概率  $P_j$  很大, 可是它的学习率却很小。当  $\beta = 1$  时, 如果抽样概率  $P_j$  变大 10 倍, 则学习率  $\alpha_j$  减小 10 倍。抽样概率、学习率两者岂不是抵消了吗? 优先经验回放有什么意义呢?

两者其实并没有抵消, 因为下面两种方式并不等价:

- 设置学习率为  $\alpha$ , 使用样本  $(s_j, a_j, r_j, s_{j+1})$  计算一次梯度, 更新一次参数  $w$ ;
  - 设置学习率为  $\frac{\alpha}{10}$ , 使用样本  $(s_j, a_j, r_j, s_{j+1})$  计算十次梯度, 更新十次参数  $w$ 。
- 乍看起来两种方式区别不大, 但其实第二种方式是对样本更有效的利用。第二种方式的缺点在于计算量大了十倍; 所以第二种方式只被用于重要的样本。

# 双Q网络 (Double DQN)

回顾一下DQN中的TD目标:  $y = r + \gamma \max_{a'} Q(s', a'; \omega)$ 。它包含一个最大化算子  $\max$  的操作。而  $Q$  又由于环境、非稳态、函数近似或者其他原因, 可能带有噪声。因此, 下一个  $Q$  值往往被过估计了。

解释一下为什么最大化会导致过估计 (高估)。设  $x_1, \dots, x_d$  为任意  $d$  个实数。往  $x_1, \dots, x_d$  中加入任意均值为零的随机噪声, 得到  $Z_1, \dots, Z_d$ , 它们是随机变量, 随机性来源于随机噪声。很容易证明均值为零的随机噪声不会影响均值:

$$\mathbb{E}[\text{mean}(Z_1, \dots, Z_d)] = \text{mean}(x_1, \dots, x_d).$$

用稍微复杂一点的证明, 可以得到:

$$\mathbb{E}[\max(Z_1, \dots, Z_d)] \geq \max(x_1, \dots, x_d).$$

公式中的期望是关于噪声求的。这个不等式意味着先加入均值为零的噪声, 然后求最大值, 会产生高估。

如果所有的Q值都被高估了相同的幅度, 那么在决策时也没有影响。但在实际情况下高估量并不是均匀的 (考虑对经验回放数组的采样), 这会增加网络预测的偏差。

# 双Q网络 (Double DQN)

标准DQN的学习目标可以被写为如下式子:

$$y = r + \gamma Q \left( s', \arg \max_{a'} Q(s', a'; \omega); \omega \right)$$

而采用了目标网络的DQN的TD目标为:

$$y = r + \gamma Q \left( s', \arg \max_{a'} Q(s', a'; \omega^-); \omega^- \right)$$

它们都没有切断max带来的高估问题:  $\omega/\omega^-$  既用于估计  $Q$  值, 又用于对估计过程中的下一个动作  $a'$  进行选择。而 Double DQN 的核心思想是在这两个阶段使用两个不同的网络, 以去除选择和评价中噪声的相关性。因此, 一个很自然的选择就成了:

$$y = r + \gamma Q \left( s', \arg \max_{a'} Q(s', a'; \omega); \omega^- \right)$$

# 双Q网络 (Double DQN)

选择

求值

自举造成偏差

最大化造成高估

原始 Q 学习

DQN

DQN

严重

严重

智能算法及应用@华南理工大学2025

Q 学习 + 目标网络

目标网络

目标网络

不严重

严重

双 Q 学习

DQN

目标网络

不严重

不严重



# 对决网络 (Dueling DQN)

将动作价值函数  $Q$  分解为状态值函数  $V$  和优势函数 (Advantage Function)  $A$ , 以提高函数近似能力。

$$Q(s, a) = V(s) + A(s, a)$$

智能算法及应用@华南理工大学2025

- $V(s)$  是对不同的状态, 它都有一个值。
- $A(s, a)$  它是对不同的状态, 不同的动作都有一个值。

优势函数  $A$  定义为  $Q$  与  $V$  的差, 表现在状态  $s$  下采取当前行动  $a$  能获得期望累计奖励  $Q(s, a)$  和平均表现  $V(s)$  之间的区别: 如果优于平均表现, 那么优势函数为正, 反之则为负。

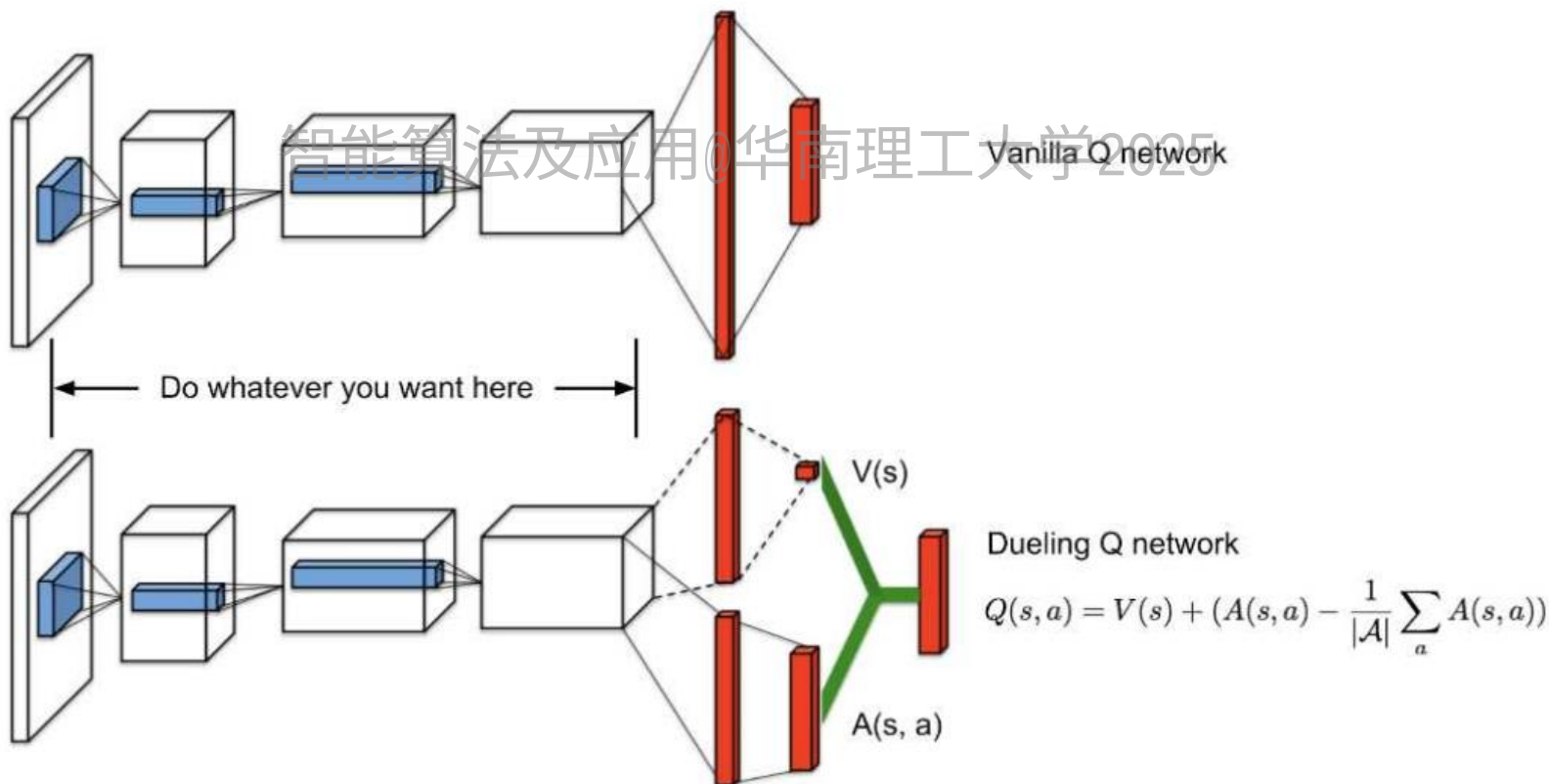
# 对决网络 (Dueling DQN)

为了解决优势的不确定性（例如2可以分解为0+2或者1+1），需要为  $A$  添加约束，通常要求一个状态的  $A$  的期望为 0。让每个  $A$  值减去当前状态所有动作对应  $A$  值的平均值，可以保证期望值为 0 的约束，增加输出的稳定性。

$$Q(s, a; \mathbf{w}) = V(s; \mathbf{w}^V) + \left( A(s, a; \mathbf{w}^A) - \frac{1}{|A|} \sum_{a'} A(s, a'; \mathbf{w}^A) \right)$$

# 对决网络 (Dueling DQN)

Dueling DQN 与 DQN 区别主要在网络结构: DQN 给定一个状态给出的是状态下各动作  $Q$  值的估计; Dueling DQN 在最后分为两部分, 一部分预测这个状态的  $V$  值 (单个值, 代表这个状态的好坏), 另一部分预测各动作对应的  $A$  (代表各动作对当前状态的改善), 最后加和在一起作为对  $Q$  的预测。



# 多步DQN (Multi-Step DQN)

原始的 DQN 使用的是当前的即时奖励和下一时刻的价值估计作为目标价值, 在训练前期策略偏差较大的情况下目标价值偏差大, 训练慢不稳定。

通过 multi-step 考虑更深几步的 reward 获得对目标价值更准确的估计, 加快训练速度。此时TD目标变为:

$$y_t = \sum_{i=0}^{m-1} \gamma^i r_{t+i} + \gamma^m \max_{a'} Q(s_{t+m}, a')$$

单步TD目标:

$$\hat{y}_t = r_t + \gamma \hat{q}_{t+1}.$$

(自举)

$\xleftarrow{m=1}$

$m$ 步TD目标:

$$\hat{y}_t = \sum_{i=0}^{m-1} \gamma^i r_{t+i} + \gamma^m \hat{q}_{t+m}.$$

$\xrightarrow{m=n-t+1}$

观测到的回报:

$$u_t = \sum_{i=0}^{n-t} \gamma^i r_{t+i}.$$

(蒙特卡洛)

# 噪声DQN (Noisy DQN)

- Noise on Action (Epsilon Greedy)

$$a = \begin{cases} \arg \max_a Q(s, a), & \text{with probability } 1 - \varepsilon \\ \text{random}, & \text{otherwise} \end{cases}$$

智能算法及应用@华南理工大学2025

- Noise on Parameters      Inject noise into the parameters of Q-function **at the beginning of each episode**

$$a = \arg \max_a \tilde{Q}(s, a)$$

$$Q(s, a) \xrightarrow{\text{Add noise}} \tilde{Q}(s, a)$$

The noise would **NOT** change in an episode.

# 噪声DQN (Noisy DQN)

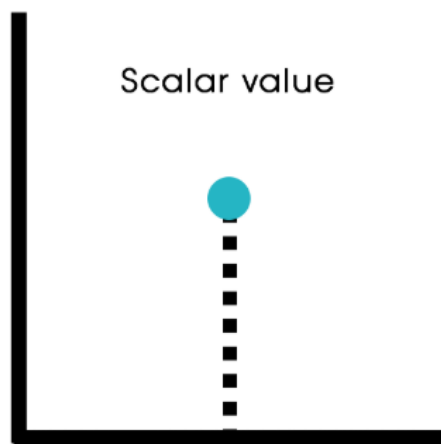
| 对比维度      | $\epsilon$ -Greedy | NoisyNet                      |
|-----------|--------------------|-------------------------------|
| 噪声注入位置    | 动作空间（直接扰动动作选择）     | 参数空间（通过噪声扰动神经网络权重）            |
| 噪声更新频率    | 每时间步独立决定是否探索       | Episode开始时重置噪声，episode内保持固定   |
| 探索的连贯性    | 低（逐时间步独立随机扰动）      | 高（一个episode内噪声固定，形成方向一致的探索轨迹） |
| 探索与策略的耦合性 | 解耦（噪声独立于策略优化）      | 强耦合（噪声是网络参数的一部分，与策略共同优化）      |
| 学习稳定性     | 低（频繁动作切换导致Q值震荡）    | 高（固定噪声下参数更新方向更一致）             |

在训练的时候往 DQN 的参数中加入噪声，不仅有利于探索，还能增强鲁棒性（即使参数被扰动，DQN 也能对Q值做出可靠的估计）。

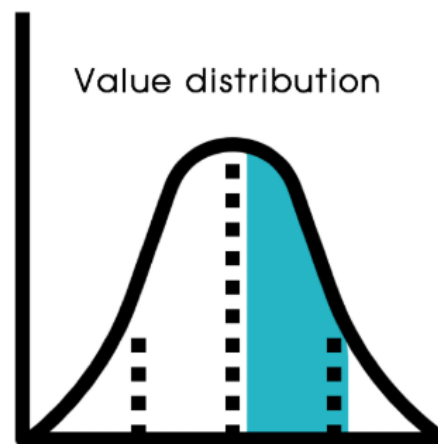
# 分布DQN (Distributional DQN)

把传统DQN中的value function换成了**value distribution**。

- 原来的DQN中的值函数是  $Q(s, a)$ ，它接受一个  $(s, a)$ ，输出一个实数，这个实数就是这个状态动作对  $(s, a)$  的评估。
- Distributional DQN中的值分布，它接受一个  $(s, a)$ ，输出一个分布，这个分布描绘了状态动作对  $(s, a)$  的所有取值的可能性。

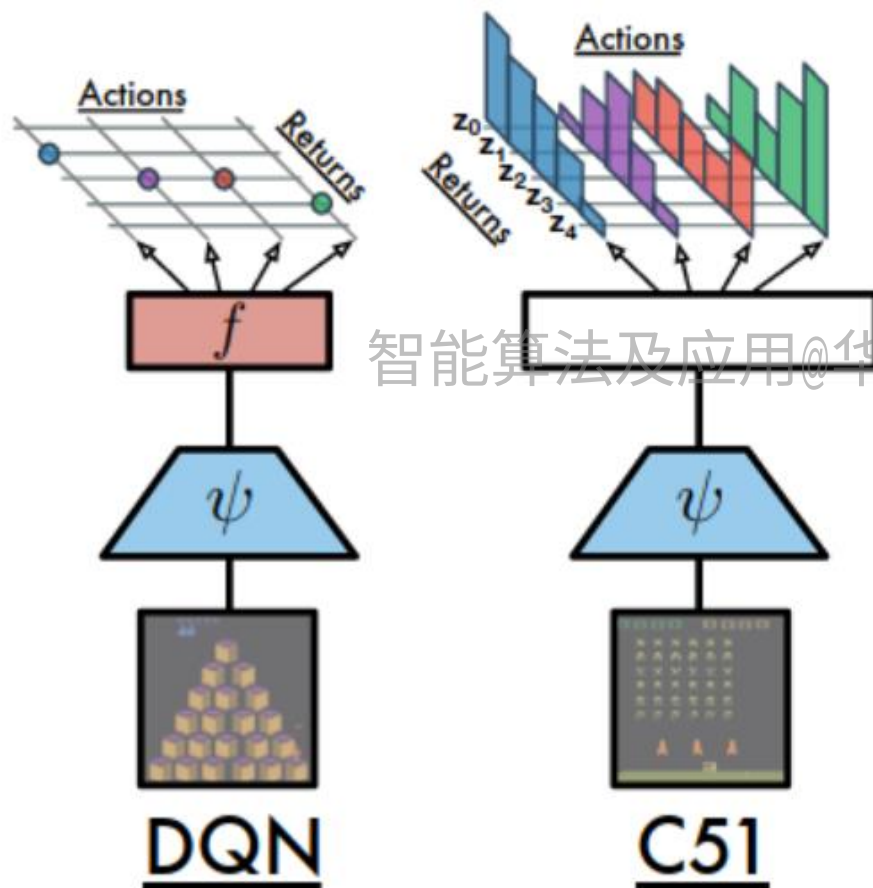


General RL



Distributional RL

# 分布DQN (Distributional DQN)

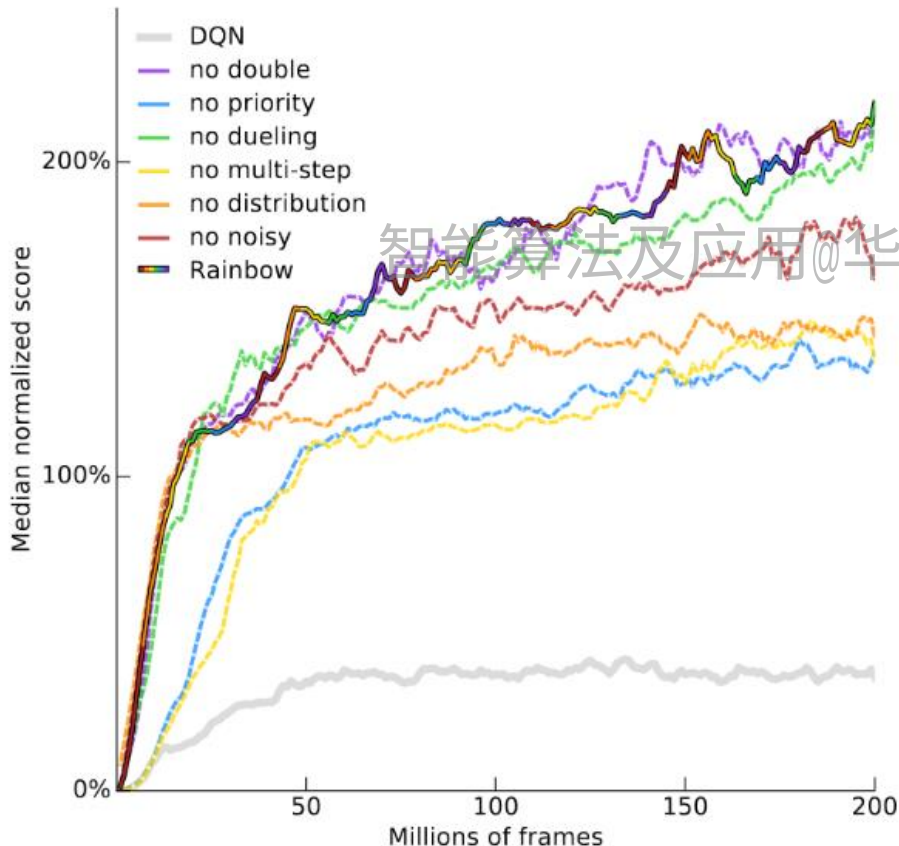


- 表示分布有很多方法，但是我们很快就抛弃掉了高斯分布这种参数化的方法，因为它是单峰的。我们使用 Distributional DQN 就是希望它能实现多个峰值的值分布，从而学习到更好的结果。
- C51 算法选择直方图来表示对于价值分布的估计，并将价值限定在  $[V_{\min}, V_{\max}]$  之间。在  $[V_{\min}, V_{\max}]$  选择  $N = 51$  个等距的价值采样点。网络的输出便是这  $N$  个价值采样点的概率。



# Rainbow

Rainbow 将上述六种 DQN 结合:



- 优先经验回放和多步TD作用对效果的提升作用是最大的;
- 再接下来是distributional loss;
- 再接下来是noisy net;
- 对于对决网络和双Q网络,发现它们的总体效果其实并不明显,尤其是后者,在少数游戏中能带来效果提升,在少数游戏中也带来效果下降。